MODULE 14 · BLOCK 3 - DATA COLLECTIONS

Dictionaries

Keys and values, which keys are allowed, KeyError versus get(), changing a dictionary, and looping over it.

BLOCK 3 of 4 EXAM WEIGHT 25% MODULE 14 of 28

admin@sthithapragna.com

A dictionary maps **keys** to values. You look up by key, never by position, and each key appears **only once**.

Braces, colons, commas

```
ages = {"Ann": 31, "Bob": 27}
empty = {}
pairs = dict([("x", 1), ("y", 2)])
print(ages["Ann"], len(ages), type(empty))   # 31 2 <class 'dict'>
print(pairs)                                 # {'x': 1, 'y': 2}
```

1. Each entry is key colon value. len counts the pairs.
2. Empty braces make an empty dictionary, not a set.
3. dict() turns a sequence of two-element pairs into a dictionary.

Keys must be hashable. Values can be anything

TYPE	AS A KEY?	WHY
`str, int, float, bool`	yes	immutable values
`tuple, e.g. (1, 2)`	yes	immutable, as long as its contents are
`list, e.g. [1, 2]`	no: TypeError	a list can change, so it cannot be hashed
`dict`	no: TypeError	also changeable

ON THE EXAM

Python 3.14 words the error as: cannot use 'list' as a dict key (unhashable type: 'list'). Still a TypeError.

1, 1.0 and True are the same key

```
1  d = {1: "int", 1.0: "float", True: "bool"}
2  print(d, len(d))            # {1: 'bool'} 1
3  d2 = {"a": 1, "a": 2}
4  print(d2)                   # {'a': 2}
```

1-2 1 == 1.0 == True, so all three land in one entry. The first key stays; the last value wins.

3-4 Repeating a key in a literal is legal. The later value replaces the earlier one.

Square brackets fail. get() does not

D[KEY]

KeyError if missing

```
1  d = {"a": 1}
2  print(d["z"])
```

KeyError: 'z'. The error names the missing key.

D.GET(KEY, DEFAULT)

None, or your default

```
1  d = {"a": 1}
2  print(d.get("z"), d.get("z", 0))
```

None 0. get never raises; it returns None unless you give a default.

THE TRAP

d[0] on a dictionary is a key lookup, not a position. With no key 0, it is a KeyError.

Assigning to a key adds it, or replaces its value

```
1  stock = {"tea": 5}
2  stock["cake"] = 2          # new key: added
3  stock["tea"] = 7           # existing key: value replaced
4  stock["tea"] += 1          # read the value, then write it back
5  stock.update({"cake": 3, "jam": 1})
6  print(stock)               # {'tea': 8, 'cake': 3, 'jam': 1}
```

2-3 The same syntax adds a new key or replaces an existing value.

4 += must read first, so the key must already exist. stock["milk"] += 1 is a KeyError.

5 update() adds or replaces several entries at once.

del, pop, popitem and clear

```
1  d = {"a": 1, "b": 2, "c": 3}
2  del d["a"]                  # removes the pair
3  print(d.pop("b"))           # 2: removes and returns the value
4  print(d.popitem())          # ('c', 3): the most recently added pair
5  d.clear()
6  print(d)                    # {}
```

3 pop(key) returns the value. pop(key, default) avoids a KeyError.

4 popitem() removes and returns the last pair added, as a tuple.

del d["q"] and d.pop("q") on a missing key raise KeyError. del d removes the whole name.

in checks the keys, not the values

```
1  d = {"Ann": 7, "Bob": 9}
2  print("Ann" in d, 7 in d)          # True False
3  print(7 in d.values())             # True
4  print(("Bob", 9) in d.items())     # True
```

2 in on a dictionary only ever looks at keys. 7 is a value, so it is not found.

3-4 To search values or pairs, ask values() or items() instead.

Keys by default; values() and items() on request

```
1  d = {"Ann": 7, "Bob": 9}
2  for k in d:                  # keys: Ann, Bob
3      print(k, d[k])
4  for k, v in d.items():       # pairs, unpacked
5      print(k, v)
6  print(list(d.keys()), list(d.values()))   # ['Ann', 'Bob'] [7, 9]
7  print(sorted({"z": 1, "x": 2}))           # ['x', 'z']
```

2-3 Looping over the dictionary itself gives the keys. Look values up with d[k].

4-5 items() gives each pair as a tuple, unpacked into two names.

7 sorted() of a dictionary returns its keys, sorted, as a list.

Printed directly, the views look like dict_keys(['Ann', 'Bob']). Wrap them in list() to get a plain list.

Dictionaries remember insertion order

```
1  d = {"b": 1, "a": 2}
2  d["c"] = 3
3  d["b"] = 9                  # replacing keeps b's place
4  print(list(d))              # ['b', 'a', 'c']
5  del d["b"]
6  d["b"] = 0                  # re-added: goes to the end
7  print(list(d))              # ['a', 'c', 'b']
```

3 Changing a value does not move its key.

5-6 A deleted key that is added again counts as new, and goes to the end.

Chained lookups: the value becomes the next key

```
1  data = {"one": "two", "two": "three", "three": "one"}
2  res = data["one"]              # "two"
3  for _ in range(2):
4      res = data[res]             # "three", then "one"
5  print(res)                      # one
```

4 Each pass looks up the current value as a key. Write down res after every pass.

3 The underscore is an ordinary name, used when the loop variable is not needed.

Same rules as lists

D2 = D

Two names, one dictionary

```
1  d = {"a": 1}
2  d2 = d
3  d2["a"] = 5
4  print(d, d2 is d)
```

{'a': 5} True. The change shows through both names.

D3 = D.COPY()

A separate dictionary

```
1  d = {"a": 1}
2  d3 = d.copy()
3  d3["a"] = 5
4  print(d, d3 is d)
```

{'a': 1} False. The copy changes alone.

ON THE EXAM

copy() is shallow, as for lists: a list stored as a value is still shared.

What goes wrong

CODE, WITH D = {"A": 1}	ERROR
`d["z"] d[0] del d["z"]`	KeyError
`d["a", "b"]`	KeyError: ('a', 'b'): the commas make one tuple key
`{[1, 2]: "x"}`	TypeError: a list is unhashable
`d.vals()`	AttributeError. Did you mean: 'values'?
`deleting keys inside for k in d`	RuntimeError: dictionary changed size during iteration

THE TIEBREAKER

Missing key: KeyError. Unhashable key: TypeError. Misspelled method: AttributeError.

Related things you may see

- **{}** is an empty dictionary. **{1, 2}**, with no colons, is a **set**: not on PCEP-30-02.
- Dictionary comprehensions: {k: k * 2 for k in range(3)}.
- Python 3.9 added | to merge dictionaries: a | b.
- For the exam: literals, dict(), [], get, update, del, pop, popitem, clear, keys, values, items, copy.

What to carry into the exam

- No positions: d[0] looks for the **key** 0. Missing key: **KeyError**.
- 1, 1.0 and True are **one key**: first spelling kept, last value wins.
- get() returns None or a default; pop() returns the value; popitem() the last pair.
- in and for look at **keys**. Use values() or items() for the rest.
- Insertion order is kept. d2 = d is an alias; copy() is a shallow copy.

What is the output of this program?

```
1  data = {1: 3, 2: 1, 3: 2}
2  x = 1
3  for _ in range(len(data)):
4      x = data[x]
5  print(x)
```

A 1

B 2

C 3

D The code raises a KeyError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = {1: 3, 2: 1, 3: 2}
2  x = 1
3  for _ in range(len(data)):
4      x = data[x]
5  print(x)
```

Three passes: x = 1 -> 3 -> 2 -> 1.

A. 1
1 goes to 3, 3 goes to 2, 2 goes back to 1.

B. 2
Stops one pass early. len(data) is 3.

C. 3
Stops after the first pass.

D. The code raises a KeyError
Every value is also a key, so every lookup succeeds.

THE SIGNAL

For chained lookups, write the value after every pass. Count passes from len().

What is the output of this program?

```
1  d = {True: "yes", 1: "one", 1.0: "float"}
2  print(len(d), d[1])
```

A 3 one

B 1 float

C 1 yes

D 3 float

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  d = {True: "yes", 1: "one", 1.0: "float"}
2  print(len(d), d[1])
```

True == 1 == 1.0, so one entry. d is {True: 'float'}.

A `3 one`

Treats the three keys as different. They are equal.

B `1 float`

One entry; each assignment replaced the value, so the last one stays.

C `1 yes`

Keeps the first value. The first key spelling stays, but the last value wins.

D `3 float`

Only one entry exists, so len is 1.

THE SIGNAL

Equal keys collide: one entry, the first key's spelling, the last value.

What is the output of this program?

```
stock = {"tea": 0, "jam": 4}
print(stock.get("tea", 9), stock.get("cake", 9),
      "jam" in stock, 4 in stock)
```

A 9 9 True False

B 0 9 True True

C 0 None True False

D 0 9 True False

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  stock = {"tea": 0, "jam": 4}
2  print(stock.get("tea", 9), stock.get("cake", 9),
3        "jam" in stock, 4 in stock)
```

A `9 9 True False`

tea exists, so get returns its value, 0, even though 0 is falsy.

B `0 9 True True`

4 is a value, and in only checks keys.

C `0 None True False`

A default was given, so a missing key returns 9, not None.

D `0 9 True False`

Existing key: its value. Missing key: the default. in checks keys.

THE SIGNAL

get uses the default only when the key is missing, never because the value is falsy.

What is the output of this program?

```
1  d = {"a": 1, "b": 2}
2  d["c"] = d.pop("a") + d["b"]
3  d["b"] = 5
4  print(d)
```

A `{'a': 1, 'b': 5, 'c': 3}`

B `{'c': 3, 'b': 5}`

C `{'b': 5, 'c': 3}`

D The code raises a KeyError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  d = {"a": 1, "b": 2}
2  d["c"] = d.pop("a") + d["b"]
3  d["b"] = 5
4  print(d)
```

pop removes a and returns 1. 1 + 2 is 3, added as c. Replacing b keeps its place.

A `{'a': 1, 'b': 5, 'c': 3}`
pop removes the key as well as returning its value.

B `{'c': 3, 'b': 5}`
Replacing b's value does not move b to the end.

C **`{'b': 5, 'c': 3}`**
a removed, c added at the end, b updated in place.

D The code raises a KeyError
a and b both exist when they are read.

THE SIGNAL

pop removes as it returns. Updating a value keeps its key's position.

What is the output of this program?

```
menu = {"tea": 2.5, "cake": 3.75}
for item, price in menu.items():
    print(item[0] + str(price)[0], end=" ")
```

A t2 c3

B tea2 cake3

C 2t 3c

D t2.5 c3.75

◆ PAUSE AND DECIDE

What is the output of this program?

```
menu = {"tea": 2.5, "cake": 3.75}
for item, price in menu.items():
    print(item[0] + str(price)[0], end=" ")
```

"tea"[0] is "t"; str(2.5)[0] is "2". Then "c" and "3".

A `t2 c3`

items() gives each pair; [0] takes a first character of each part.

B `tea2 cake3`

Forgets the [0] on the item.

C `2t 3c`

Reverses the order of the join.

D `t2.5 c3.75`

Forgets the [0] on the price.

THE SIGNAL

items() unpacks into key and value. Convert to str before indexing a number.

> d = {"a": 1} has been assigned. Each line is run on its own.

Which two lines raise an exception? (Choose two.)

A `print(d["b"])`

B `print(d.get("b"))`

C `print(d.pop("a", 0))`

D `e = {[1, 2]: "x"}`

◆ PAUSE AND DECIDE

Which two lines raise an exception? (Choose two.)

A `print(d["b"])`

b is not a key. KeyError.

B `print(d.get("b"))`

get never raises. Prints None.

C `print(d.pop("a", 0))`

a exists, so it prints 1. The default would only matter if it did not.

D `e = {[1, 2]: "x"}`

A list cannot be a key. TypeError.

THE SIGNAL

Square brackets on a missing key raise; get does not. Keys must be hashable.